IT1244

Artificial Intelligence: Technology & Impact

Week 8 Tutorial 6

TODAY'S AGENDA

1. Recap of concepts
2. Discussion of tutorial questions
3. Pollev Questions

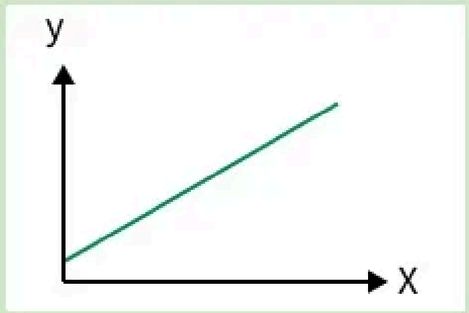
RECAP

LOGISTIC REGRESSION

Logistic regression is a linear model for classification that predicts probabilities using the sigmoid function.

Linear Regression

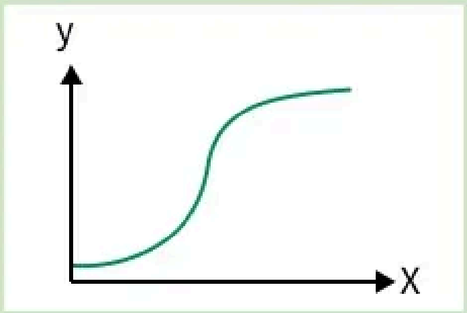
- Predicts continuous values
- Uses best-fit line
- Solves regression problems



vs

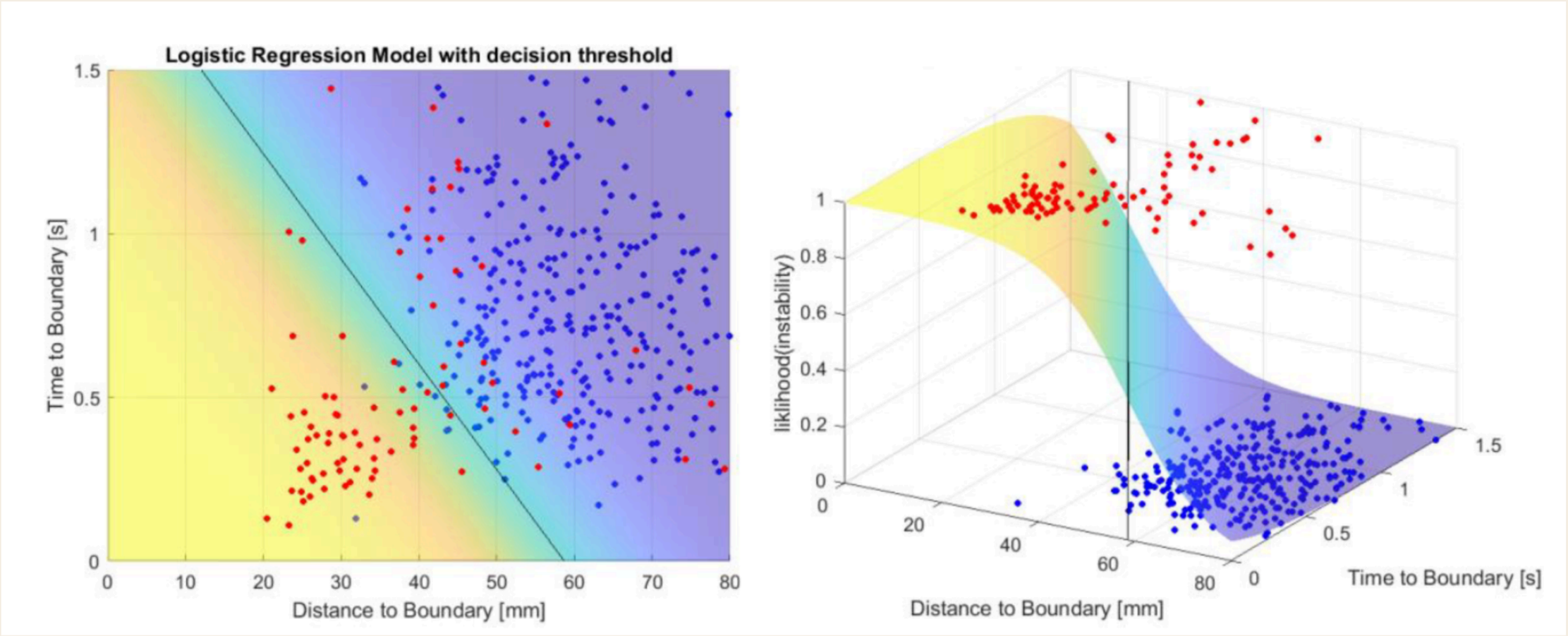
Logistic Regression

- Predicts categorical classes
- Uses sigmoid S-curve
- Solves classification problems



LET'S TRY TO UNDERSTAND LOG REG

TLDR: We want to find a hyperplane that best separates the two classes.



LET'S TRY TO UNDERSTAND LOG REG

Decision Boundary / Separating Hyperplane:

$$w^T x + b = 0$$

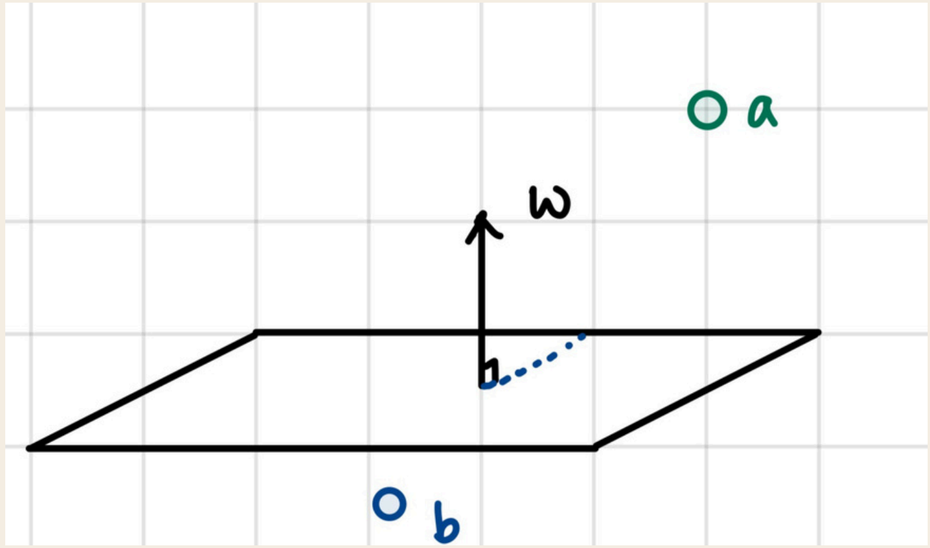
1. If a point x is above the hyperplane (eg. point A),

$$w^T x + b > 0$$

2. Else if a point x is below the hyperplane (eg. point B),

$$w^T x + b < 0$$

3. The further the distance from the plane, the greater the magnitude of: $|w^T x + b|$



LET'S TRY TO UNDERSTAND LOG REG

We know: Sample that are farther from the decision boundary

→ Larger $|w^T x + b|$

Question:Should samples farther from the decision boundary have higher or lower predicted probability of belonging to a class?

Answer: Samples farther from the decision boundary should have **higher predicted probability (higher confidence)** of belonging to their class.

Samples closer to the boundary have lower confidence, since the model is more uncertain.

Logistic Regression leverages this idea exactly!

LET'S TRY TO UNDERSTAND LOG REG

Logistic Regression:

Converts $w^T x + b$ into a probability using sigmoid function

Sigmoid Function: $$g(z) = \frac{1}{(1 + e^{-z})}$$

Sigmoid function maps $w^T x + b$ (-∞, ∞) to probability [0, 1].

Quick Question: If a point has g(z) = 0.5, does it lie above, on, or below the decision boundary?

LOGISTIC REGRESSION STEPS

Step 1: Applying the regression function

$$z = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

Step 2: Passing the output into the sigmoid function to get probability [0,1]

$$g(z) = \frac{1}{(1 + e^{-z})}$$

Step 3: Choose a cutoff value to turn probability into a classifier

- Points with estimated probability ≥ threshold belongs to the “positive class” (label = 1), else “negative class” (label = 0)

COST FUNCTION (LOGISTIC REGRESSION)

Cost of a single instance:

$$\text{cost}(\hat{y}(x), y) = \begin{cases} -\log(\hat{y}(x)) & \text{if } y = 1, \\ -\log(1 - \hat{y}(x)) & \text{if } y = 0. \end{cases}$$

Or:

$$\text{cost}(\hat{y}(x_i), y_i) = -y_i \log(\hat{y}(x_i)) - (1 - y_i) \log(1 - \hat{y}(x_i))$$

Idea: If the predicted probability is close to the true label, the cost should be small.

If the model is confident but wrong, the cost becomes very large.

ERROR FUNCTION (LOGISTIC REGRESSION)

Error function: Simply take average of cost across entire dataset

$$E(\mathbf{W}) = \frac{1}{2N} \sum_{i=1}^{N} \text{cost}(\hat{y}(x_i), y_i)$$

Recall: We want to minimize this error function!

Also, this error function is also convex! That means there exist a unique global minimum (which is what we hope to find through gradient descent!)

GRADIENT DESCENT (LOGISTIC REG)

Minimize error function!

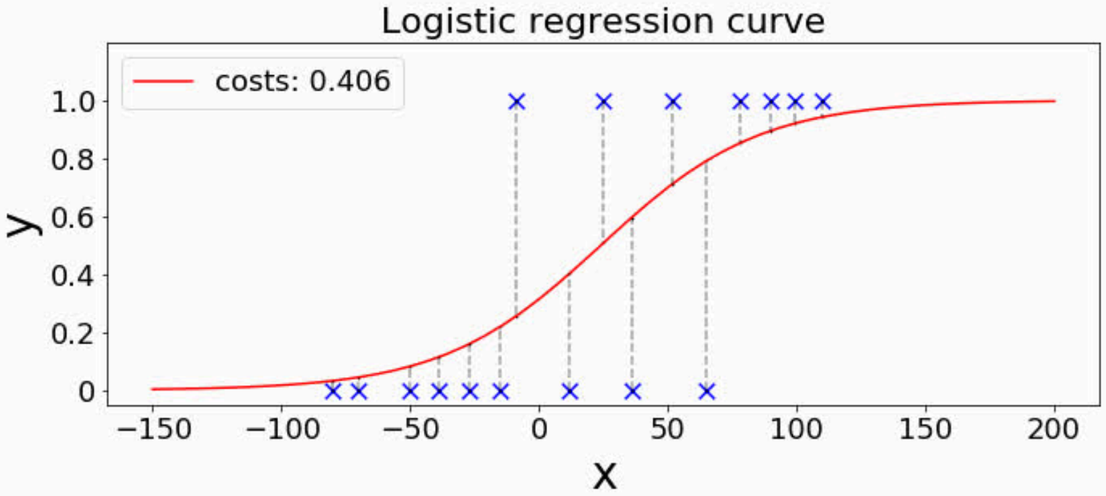
$$\frac{\partial E(W)}{\partial w_j} = -\frac{1}{N}\sum_{i=1}^{N}\left[y^{(i)} - \hat{y}^{(i)}\right] x_j^{(i)}$$

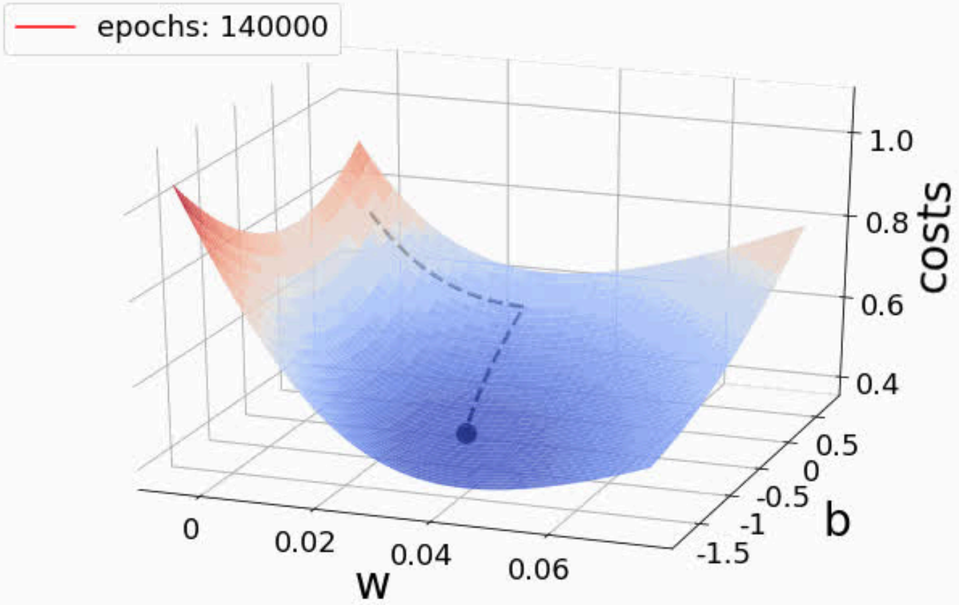
‼ Interestingly, the result looks exactly the same as linear regression.

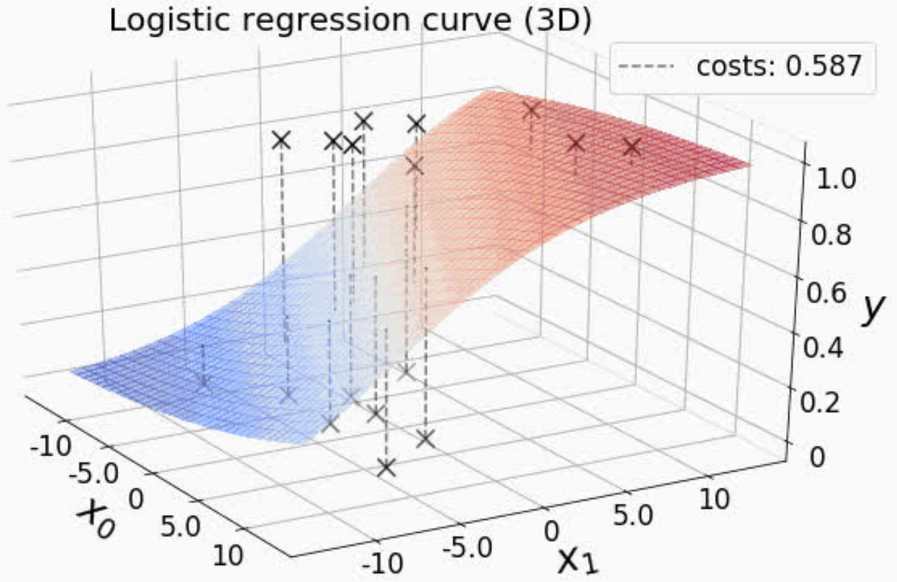
The key difference is that the prediction y_hat now comes from the sigmoid function, meaning it represents a probability.

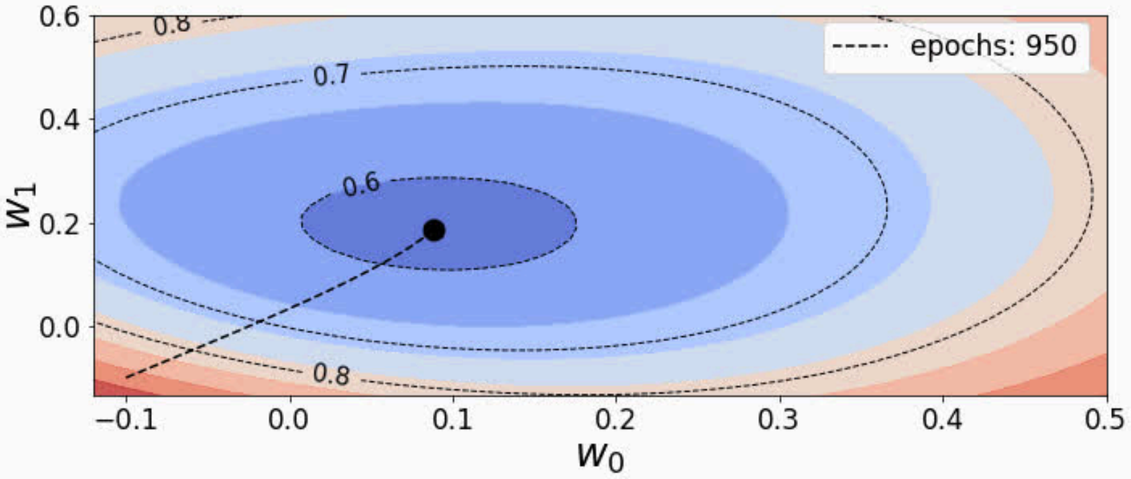
$$w_j := w_j + L\,\frac{1}{N}\sum_{i=1}^{N}(y^{(i)} - \hat{y}(x^{(i)}))x_j^{(i)}$$

GRADIENT DESCENT FOR LOG REG









CONFUSION MATRIX

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Sometimes Negative and Positive will switch places. So please read carefully before answering!

Accuracy =
(TP + TN) / (TP + TN + FP + FN)

TPR (Recall) = TP / (TP + FN)

FPR = FP / (FP + TN)

FNR = FN / (TP + FN)

Precision = TP / (TP + FP)

F1-score = 2TP / (2TP + FP + FN)

Support = TP + FN

TUTORIAL QUESTIONS

TUTORIAL QUESTION 1

Formulate logistic regression function and run the Gradient Descent algorithm for 3 iterations. Finally show the output of W vector after 3 iterations. The learning rate is L=0.01 and initial values of w's are 0.

X1	X2	Y
3.281084	2.960537	0
1.965489	2.772125	0
3.896562	4.810294	0
8.127531	3.169262	1
5.832441	2.498627	1
7.422597	2.181064	1

TUTORIAL QUESTION 1

X1	X2	Y
3.281084	2.960537	0
1.965489	2.772125	0
3.896562	4.810294	0
8.127531	3.169262	1
5.832441	2.498627	1
7.422597	2.181064	1

JUST FOLLOW THESE STEPS

1) $\hat{y}^{(i)} = \sigma(w_0 + w_1 x^{(i)})$ where $\sigma(z) = \frac{1}{1+e^{-z}}$

2) $\Delta w_0 = \frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})$

$\Delta w_1 = \frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})\, x^{(i)}$

3) $w_0 = w_0 + L\,\Delta w_0$

$w_1 = w_1 + L\,\Delta w_1$

TUTORIAL QUESTION 1

[w0, w1, w2] = [0,0] | L = 0.01 | 3 iterations

X1	X2	Y
3.281084	2.960537	0
1.965489	2.772125	0
3.896562	4.810294	0
8.127531	3.169262	1
5.832441	2.498627	1
7.422597	2.181064	1

Iteration 1

step 1: $\hat{y}^{(i)} = \sigma(w_0 + w_1 x^{(i)})$

since $w_0 = w_1 = w_2 = 0$, all $w_0 + w_1 x_1 + w_2 x_2 = 0$

$\sigma(0) = \frac{1}{1+e^0} = \frac{1}{2}$ all $y^{(i)} = \frac{1}{2}$

step 2: $\Delta w_0, \Delta w_1, \Delta w_2$

$\Delta w_0 = \frac{1}{6}\left(-\frac{1}{2}-\frac{1}{2}-\frac{1}{2}+\frac{1}{2}+\frac{1}{2}+\frac{1}{2}\right) = 0$

$\Delta w_1 = \frac{1}{6}\left(-\frac{1}{2}(3.281)-\frac{1}{2}(1.965)-\frac{1}{2}(3.897)+\frac{1}{2}(8.127)+\frac{1}{2}(5.832)+\frac{1}{2}(7.425)\right) = 1.02$

$\Delta w_2 = \frac{1}{6}\left(-\frac{1}{2}(2.961)-\frac{1}{2}(2.772)-\frac{1}{2}(4.810)+\frac{1}{2}(3.169)+\frac{1}{2}(2.499)+\frac{1}{2}(2.181)\right) = -0.2245$

TUTORIAL QUESTION 1

[w0, w1, w2] = [0,0] | L = 0.01 | 3 iterations

X1	X2	Y
3.281084	2.960537	0
1.965489	2.772125	0
3.896562	4.810294	0
8.127531	3.169262	1
5.832441	2.498627	1
7.422597	2.181064	1

Iteration 1

Step 3: Update w_0, w_1, w_2

$w_0 = 0 + 0.01(0) = 0$

$w_1 = 0 + 0.01(1.02) = 0.102$

$w_2 = 0 + 0.01(-0.2245) = -0.00225$

w0, w1 and w2 after iteration 1:

[0, 0.102, -0.00225]

TUTORIAL QUESTION 1

[w0, w1, w2] = [0,0] | L = 0.01 | 3 iterations

X1	X2	Y
3.281084	2.960537	0
1.965489	2.772125	0
3.896562	4.810294	0
8.127531	3.169262	1
5.832441	2.498627	1
7.422597	2.181064	1

Repeat for Iteration 2 and 3:

Iteration 1: [0, 0.102, -0.00225]

Iteration 2: [-0.00011, 0.019698, -0.00482]

Iteration 3: [-0.00033, 0.028564, -0.00768]

TUTORIAL QUESTION 1

Can also consider using Excel for this cause numbers is quite long

Q1

Given dataset

X1	X2	Y	L
3.281084	2.960537	0	0.01
1.965489	2.772125	0	
3.896562	4.810294	0	
8.127531	3.169262	1	
5.832441	2.498627	1	
7.422597	2.181064	1	

add Bias

	iteration	X0	X1	X2	Y	w0	w1	w2	y_hat	(y - y_hat)	(y-y_hat) * X1	(y-y_hat) * X2	delta_w0	delta_w1	delta_w2
	0					0	0	0							
		1	3.281084	2.960537	0				0.5	-0.5	-1.640542	-1.480269			
		1	1.965489	2.772125	0				0.5	-0.5	-0.982745	-1.386063			
		1	3.896562	4.810294	0				0.5	-0.5	-1.948281	-2.405147			
		1	8.127531	3.169262	1				0.5	0.5	4.063766	1.584631			
		1	5.832441	2.498627	1				0.5	0.5	2.916221	1.249314			
		1	7.422597	2.181064	1				0.5	0.5	3.711299	1.090532			
end of	1					0	0.01019953	-0.002245					0	1.019953	-0.2245
		1	3.281084	2.960537	0				0.506704	-0.506704	-1.66254	-1.500117			
		1	1.965489	2.772125	0				0.503456	-0.503456	-0.989537	-1.395643			
		1	3.896562	4.810294	0				0.507235	-0.507235	-1.976475	-2.439952			
		1	8.127531	3.169262	1				0.518936	0.481064	3.909859	1.524616			
		1	5.832441	2.498627	1				0.513466	0.486534	2.837678	1.215666			
		1	7.422597	2.181064	1				0.517695	0.482305	3.579954	1.051938			
end of	2					-0.0001125	0.01969776	-0.0048175					-0.011249	0.949823	-0.257249
		1	3.281084	2.960537	0				0.512561	-0.512561	-1.681756	-1.517456			
		1	1.965489	2.772125	0				0.506312	-0.506312	-0.99515	-1.40356			
		1	3.896562	4.810294	0				0.513364	-0.513364	-2.000353	-2.46943			
		1	8.127531	3.169262	1				0.536115	0.463885	3.770236	1.470172			
		1	5.832441	2.498627	1				0.525662	0.474338	2.766551	1.185195			
		1	7.422597	2.181064	1				0.533845	0.466155	3.460078	1.016713			
end of	3					-0.0003256	0.02856377	-0.0076814					-0.02131	0.886601	-0.286394

TUTORIAL QUESTION 2

Find out the other metrics to measure the performance of classification output - precision, recall, F1-score and Support

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Sometimes Negative and Positive will switch places. So please read carefully before answering!

Accuracy =
(TP + TN) / (TP + TN + FP + FN)

TPR (Recall) = TP / (TP + FN)

FPR = FP / (FP + TN)

FNR = FN / (TP + FN)

Precision = TP / (TP + FP)

F1-score = 2TP / (2TP + FP + FN)

Support = TP + FN

TUTORIAL QUESTION 2

When to use precision vs recall vs F1-score?

- **Precision:** when the cost of false positives is high
  - e.g. spam email detection - important emails classified as a spam
- **Recall:** when the cost of false negatives is high
  - e.g. cancer diagnosis negative even though you have cancer
- **F1-score:** when both false positives and negatives are important

  (*model only gets a high F1 if both precision and recall are reasonably high*)

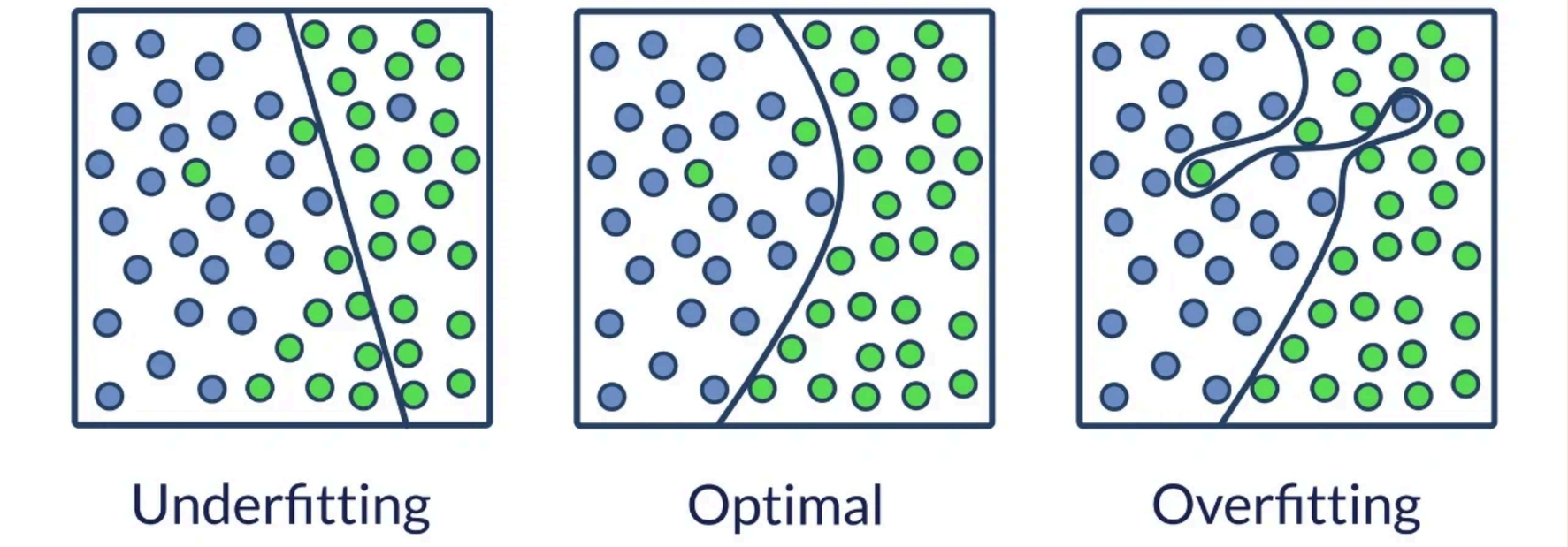
$$F1 = \frac{2(\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$$

TUTORIAL QUESTION 3

You are training a ML model and you find that your training error is close to 0, but the testing error is very high. What can be done to improve this situation?

Is this overfitting or underfitting?

Train Error Low + Test Error High = **Overfitting!**



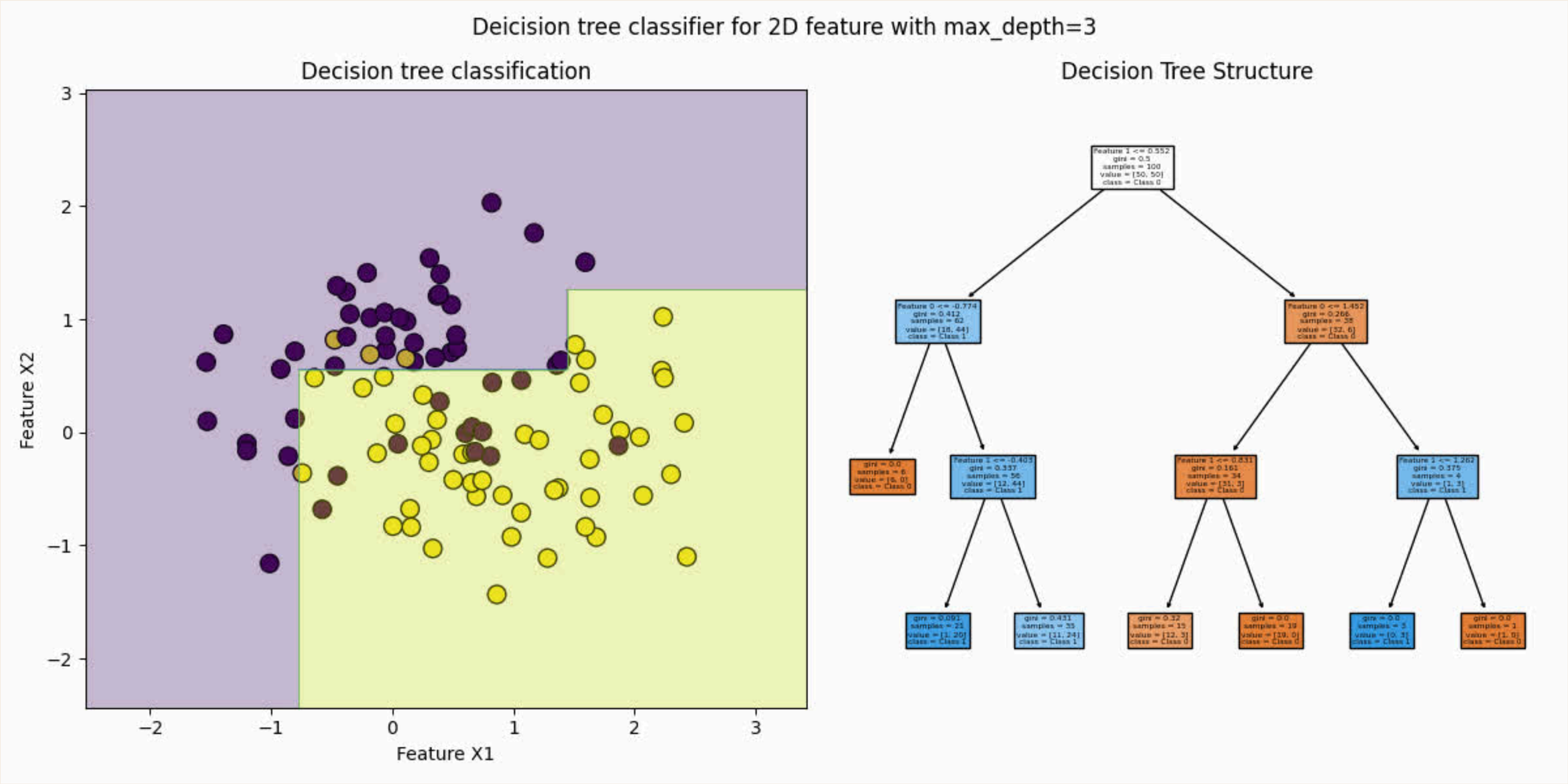
TUTORIAL QUESTION 3

You are training a ML model and you find that your training error is close to 0, but the testing error is very high. What can be done to improve this situation?

Techniques to fight underfitting and overfitting	
Underfitting	**Overfitting**
More complex model	More simple model
Less regularization	More regularization
Larger quantity of features	Smaller quantity of features
More data can't help	More data can help

TUTORIAL QUESTION 3

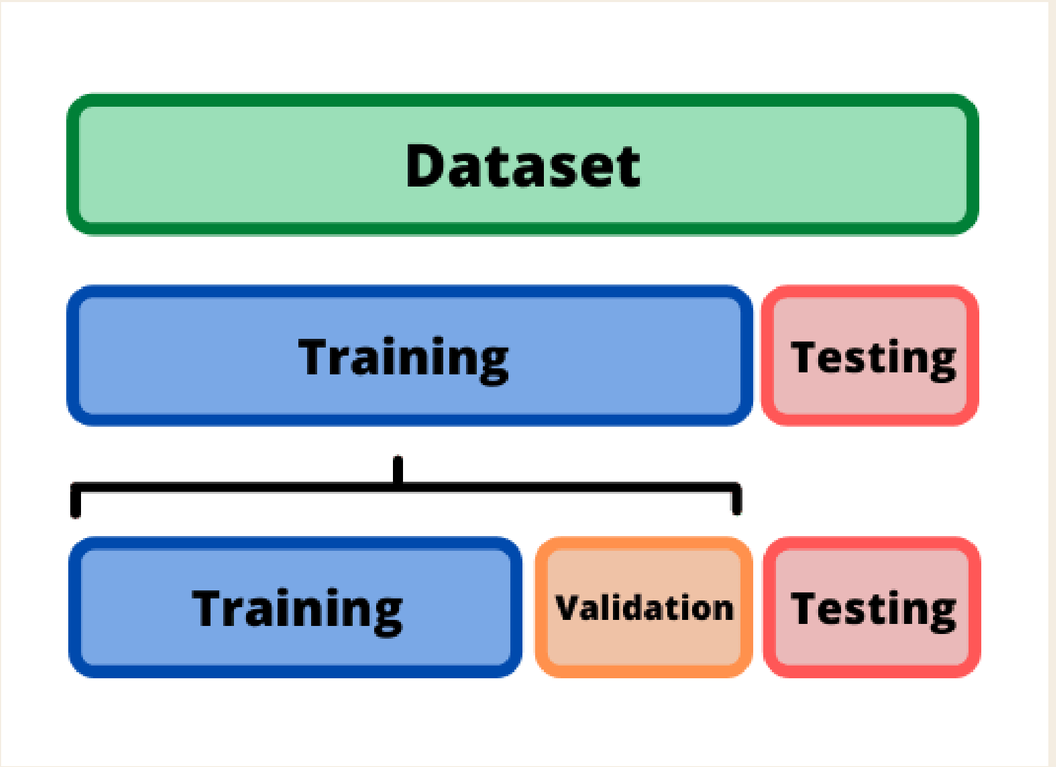
Eg. Depth of decision tree



TUTORIAL QUESTION 4

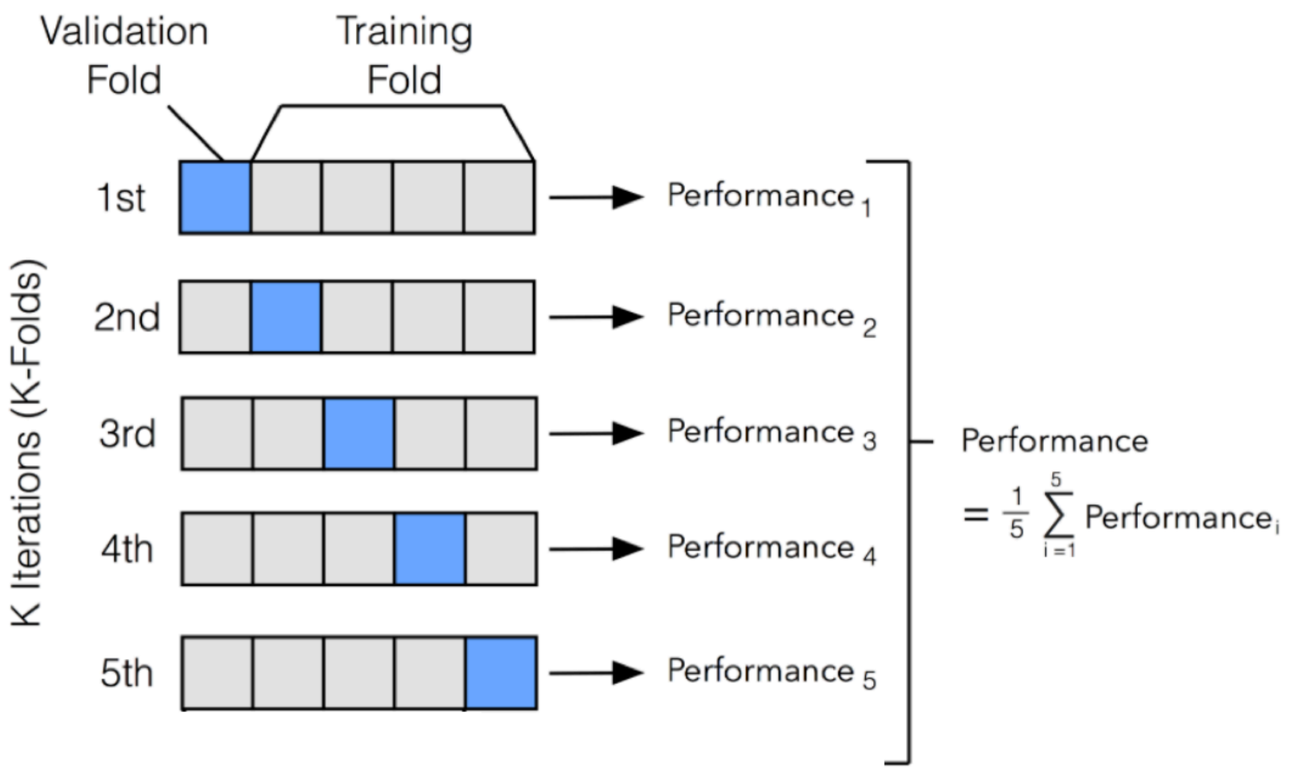
We have seen “train-and-test-split” method to split the dataset so that we avoid overfitting in our model. There are two other ways to split your dataset and train them: **k-Fold Cross-Validation** and **Leave-one-out Cross-Validation**. Discuss them.

Usual Train Validation Test Split →



TUTORIAL QUESTION 4

k-fold cross validation (eg: k = 5)



→ Split training data into k folds

→ For each fold i:

- Train on k – 1 folds
- Validate on fold i

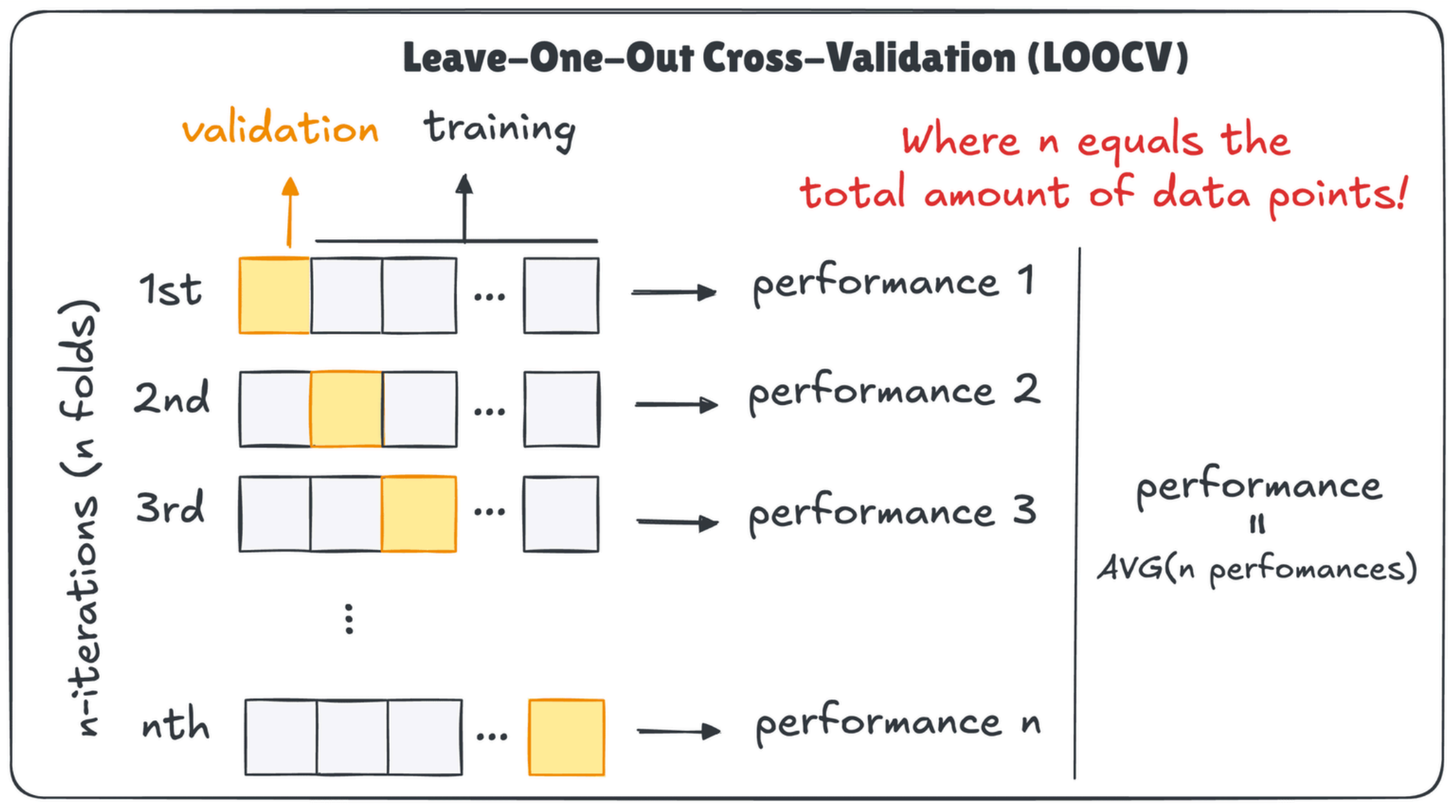
→ Repeat for k iterations

→ Average the validation metrics

Why? To provide a more reliable estimate of model performance, since a single train-test split may result in an unlucky data split.

TUTORIAL QUESTION 4

Leave-one-out Cross-Validation



→ A special case of k-fold CV (k = n)

→ Leaves out **just one instance as** a separate **test set**

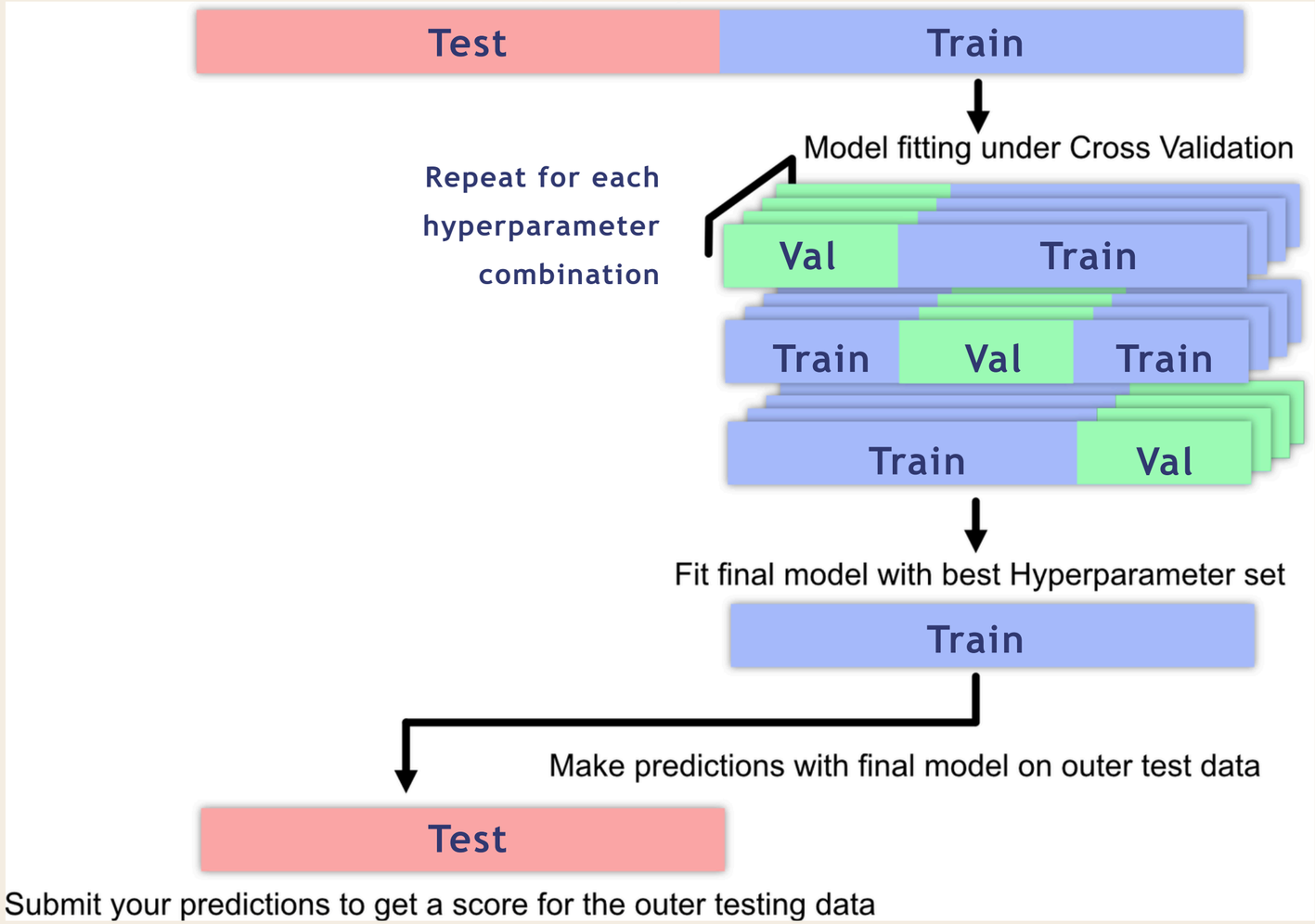
→ **Train** a model on **all the other instances** in the training set

→ **Computationally expensive**, only do this if have very **limited data.**

LOOCV uses almost the entire dataset for training in each iteration, which helps when the dataset is small because we want to leverage as much data as possible for training.

TUTORIAL QUESTION 4

The bigger picture: how Cross Validation fits in the ML pipeline



POLLEV QUESTIONS

Join by Web:

PollEv.com/kaironglee

Join by QR:

EXTRA READINGS

- What is Softmax Regression?
- What is the difference between Multiclass and Multi-label Classification?
- Multiclass Classification Models (Might be useful if you are doing multiclass classification project)

NEXT WEEK

Clustering